

1. What will be the output of the following code?

```
x = [1, 2, 3, 4, 5]
result = [i**2 for i in x if i % 2 != 0]
print(result)
```

A) [1, 4, 9, 16, 25]

B) [4, 16]

C) [1, 9, 25]

D) [2, 4]

Correct Answer: C) [1, 9, 25]

Explanation: List comprehension filters odd numbers ($i \% 2 \neq 0$) $\rightarrow$ 1,3,5 and squares them $\rightarrow$ [1,9,25]. Even numbers 2 and 4 are skipped due to if condition.

2. What will be the output of the following code?

```
data = [45, 72, 89, 55, 91, 63, 78]
total = 0
count = 0
for num in data:
    if num >= 70:
        total += num
        count += 1
print(round(total/count, 2), count)
```

A) 82.75 4

B) 82.5 4

C) 77.25 3

D) 83.5 4

Correct Answer: B) 82.5 4

Explanation: Numbers $\geq 70 \rightarrow$ 72, 89, 91, 78 $\rightarrow$ count=4. total = 72+89+91+78 = 330. $330/4 = 82.5$. $\text{round}(82.5, 2) = 82.5$.

3. What will be the output of the following code?

```
import random
```

```
import string
```

```
def generate_password(length):
```

```
    chars = string.ascii_letters + string.digits + "!@#\$"
```

```
    password = ''.join(random.choice(chars) for _ in range(length))
```

```
    return password
```

```
pwd = generate_password(8)
```

```
print(len(pwd), type(pwd))
```

A) 8 <class 'list'>

B) 8 <class 'str'>

C) 4 <class 'str'>

D) Error — random.choice cannot work on strings

Correct Answer: B) 8 <class 'str'>

Explanation:

- string.ascii_letters = a-z + A-Z (52 chars)
- string.digits = 0-9 (10 chars)
- "!@#\\$" = 4 special chars
- random.choice() picks one random character from combined string
- join() combines 8 random characters into one string
- len(pwd) = 8, type = str always regardless of random values

Q4 What will be the output of the following code?

```
fruits = ['apple', 'banana', 'cherry', 'mango']
```

```
print(fruits[1:3])
```

A) ['apple', 'banana']

B) ['banana', 'cherry', 'mango']

C) ['banana', 'cherry']

D) ['apple', 'banana', 'cherry']

Correct Answer: C) ['banana', 'cherry']

Explanation: List slicing [1:3] returns elements from index 1 up to but not including index 3.

5. Which of the following statements about Python lists is CORRECT?

A) Lists are immutable and cannot be modified after creation

B) Lists can store only elements of the same data type

C) Lists maintain insertion order and allow duplicate elements

D) Lists do not support indexing and slicing

Correct Answer: C) Lists maintain insertion order and allow duplicate elements

Explanation: Python lists are ordered, mutable, and allow duplicate values of any data type.

6. What will be the output of the following code?

```
numbers = [3, 1, 4, 1, 5, 9, 2, 6]
```

```
a = sorted(numbers)
```

```
numbers.sort(reverse=True)
```

```
print(a[0], numbers[0])
```

A) 1 9

B) 9 1

C) 1 1

D) 3 9

Correct Answer: A) 1 9

Explanation: sorted() returns a new sorted list in ascending order so a[0]=1.
sort(reverse=True) sorts original list in descending order so numbers[0]=9. Original list is modified but a remains ascending.

7.What will be the output of the following code?

```
student = {'name': 'Alice', 'age': 22, 'grade': 'A'}  
  
student['age'] = 23  
  
student['city'] = 'Hyderabad'  
  
print(len(student), student.get('grade'), student.get('score', 0))
```

- A) 3 A 0
- B) 4 A 0
- C) 4 A None
- D) 3 A None

Correct Answer: B) 4 A 0

Explanation: After updating age and adding city, dict has 4 keys. get('grade') returns 'A'.
get('score', 0) returns default value 0 since 'score' key does not exist.

8.What will be the output of the following code?

```
marks = {'Maths': 85, 'Science': 90, 'English': 78}  
  
marks['Science'] = 95  
  
del marks['English']  
  
for key, value in marks.items():  
  
    print(key, value, end=' | ')
```

- A) Maths 85 | Science 90 | English 78 |
- B) Maths 85 | Science 95 |
- C) Maths 85 | Science 95 | English 78 |
- D) Science 95 | Maths 85 |

Correct Answer: B) Maths 85 | Science 95 |

Explanation: Science value updated to 95 and English key deleted. Remaining keys printed in insertion order with updated values.

9.What will be the output of the following code?

```
t = (10, 20, 30, 40, 50)
```

```
a, b, *c = t
```

```
print(a, b, c)
```

A) 10 20 [30, 40, 50]

B) 10 20 (30, 40, 50)

C) 10 20 30

D) Error

Correct Answer: A) 10 20 [30, 40, 50]

Explanation: Tuple unpacking with * operator collects remaining elements into a list. a=10, b=20 and c captures remaining as a list [30, 40, 50].

10.What will be the output of the following code?

```
t1 = (1, 2, 3)
```

```
t2 = (3, 4, 5)
```

```
t3 = t1 + t2
```

```
print(t3.count(3), t3.index(4), len(t3))
```

A) 2 4 6

B) 1 4 6

C) 2 3 6

D) 1 3 6

Correct Answer: A) 2 4 6

Explanation: t3 = (1,2,3,3,4,5). count(3) = 2 since 3 appears twice. index(4) = 4 since 4 is at index 4. len(t3) = 6.

11.What will be the output of the following code?

```
s = {1, 2, 3, 2, 1, 4}
```

```
s.add(5)
```

```
s.discard(2)
```

```
s.discard(10)
```

```
print(len(s), 3 in s)
```

A) 4 True

B) 5 True

C) 4 False

D) Error

Correct Answer: A) 4 True

Explanation: Set removes duplicates so {1,2,3,4}. add(5) makes it {1,2,3,4,5}. discard(2) removes 2 → {1,3,4,5}. discard(10) does nothing (no error). len=4, 3 in s = True.

12.What will be the output of the following code?

```
a = {1, 2, 3, 4, 5}
```

```
b = {3, 4, 5, 6, 7}
```

```
print(a & b, a - b)
```

A) {3, 4, 5} {1, 2, 3}

B) {3, 4, 5} {1, 2}

C) {1, 2, 3, 4, 5, 6, 7} {1, 2}

D) {3, 4, 5} {6, 7}

Correct Answer: B) {3, 4, 5} {1, 2}

Explanation: & operator returns intersection (common elements) = {3,4,5}. - operator returns difference (elements in a but not in b) = {1,2}.

13.What will be the output of the following code?

```
s = "Hello Data Science"
```

```
print(s.split()[1], s.lower().count('e'), s.replace('Science', 'World'))
```

A) Data 3 Hello Data World

B) Data 2 Hello Data World

C) Hello 3 Hello Data World

D) Data 3 Hello Data Science

Correct Answer: A) Data 3 Hello Data World

Explanation: split()[1] returns second word 'Data'. lower().count('e') counts all 'e' in lowercase = 3 (hEllo, sciEncE → hello data science = 3). replace() replaces 'Science' with 'World'.

14.What will be the output of the following code?

```
s = "python programming"
```

```
print(s[7:18:2], s[::-1])
```

A) pormig gnimmargorp nohtyp

B) prgamn gnimmargorp nohtyp

C) rogrammi gnimmargorp nohtyp

D) pormig nohtyp gnimmargorp

Correct Answer: A) pormig gnimmargorp nohtyp

Explanation: `s[7:18:2]` slices from index 7 to 18 with step 2 → `'p','o','r','m','i','g' = 'pormig'`. `s[::-1]` reverses the entire string = `'gnimmargorp nohtyp'`.

15. What will be the output of the following code?

```
words = ['Python', 'is', 'fun']  
sentence = '-'.join(words)  
parts = sentence.split('-')  
print(sentence, len(parts))
```

- A) Python is fun 3
- B) Python-is-fun 3
- C) Python-is-fun 2
- D) Python is fun 2

Correct Answer: B) Python-is-fun 3

Explanation: `join()` combines list elements with '-' separator → `'Python-is-fun'`. `split('-')` splits back into `['Python','is','fun']`. `len(parts) = 3`.

16. What will be the output of the following code?

```
x = 5  
y = 10  
z = 15  
print(x < y and y < z or z < x)
```

- A) False
- B) True
- C) Error
- D) None

Correct Answer: B) True

Explanation: Operator precedence: < evaluated first → (x<y)=True, (y<z)=True, (z<x)=False. Then: (True and True) or False → True or False → True. 'and' has higher precedence than 'or'.

17.What will be the output of the following code?

```
num = 28
```

```
if num % 2 == 0:
```

```
    if num % 7 == 0:
```

```
        print("Even and Divisible by 7")
```

```
    else:
```

```
        print("Even but Not Divisible by 7")
```

```
else:
```

```
    print("Odd Number")
```

A) Odd Number

B) Even but Not Divisible by 7

C) Even and Divisible by 7

D) Error

Correct Answer: C) Even and Divisible by 7

Explanation: 28%2==0 → Even. 28%7==0 → Divisible by 7. Both conditions satisfied → prints "Even and Divisible by 7".

18.What will be the output of the following code?

```
num = 29
```

```
is_prime = True
```

```
if num < 2:
    is_prime = False
elif num == 2:
    is_prime = True
elif num % 2 == 0:
    is_prime = False
else:
    if num % 3 == 0 or num % 5 == 0 or num % 7 == 0:
        is_prime = False
```

```
if is_prime:
    print(f"{num} is Prime")
else:
    print(f"{num} is Not Prime")
```

A) 29 is Not Prime

B) 29 is Prime

C) Error

D) False

Correct Answer: B) 29 is Prime

Explanation: 29 >= 2, not equal to 2, not divisible by 2. Inner check: 29%3≠0, 29%5≠0, 29%7≠0 → is_prime remains True → prints "29 is Prime".

19.What will be the output of the following code?

a, b, c = 10, 20, 30

if a > b or b < c:

```
if a + b > c:
    print("X")
elif a * 2 == b:
    print("Y")
else:
    print("Z")
else:
    print("W")
```

A) X

B) Y

C) Z

D) W

Correct Answer: B) Y

Explanation: Outer condition: $a > b$ is False but $b < c$ ($20 < 30$) is True $\rightarrow$ enters outer if. Inner: $a + b = 30 > c = 30$ is False (not strictly greater). elif: $a * 2 = 20 == b = 20$ is True $\rightarrow$ prints "Y".

20. What will be the output of the following code?

```
char = 'E'
```

```
if char.isalpha():
    if char.lower() in 'aeiou':
        if char.isupper():
            print("Uppercase Vowel")
        else:
            print("Lowercase Vowel")
    else:
        if char.isupper():
            print("Uppercase Consonant")
        else:
            print("Lowercase Consonant")
else:
    print("Not an Alphabet")
```

- A) Lowercase Vowel
- B) Uppercase Consonant
- C) Uppercase Vowel
- D) Not an Alphabet

Correct Answer: C) Uppercase Vowel

Explanation: char='E' is alphabetic. 'e' is in 'aeiou' → it is a vowel. char.isupper() is True → prints "Uppercase Vowel".

21. What will be the output of the following code?

```
for i in range(1, 6):  
    if i == 3:  
        pass  
    print(i, end=' ')
```

- A) 1 2 4 5
- B) 1 2 3 4 5
- C) 1 2
- D) 1 2 4 5 6

Correct Answer: B) 1 2 3 4 5

Explanation: pass is a null statement — it does nothing and execution continues normally. Unlike break or continue, pass does not affect loop flow. All numbers 1 to 5 are printed.

22. What will be the output of the following code?

```
count = 0  
for i in range(1, 21):  
    if i % 2 == 0:  
        continue  
    if i % 7 == 0:  
        break  
    count += 1
```

```
print(count)
```

- A) 5
- B) 6
- C) 3

D) 7

Correct Answer: C) 3

Explanation: Loop runs 1 to 20. continue skips even numbers. Odd numbers checked: 1(count=1), 3(count=2), 5(count=3), 7 → divisible by 7 → break. Final count = 3.

23. What will be the output of the following code?

```
vowels = 0
```

```
consonants = 0
```

```
word = "Programming"
```

```
for char in word:
```

```
    if not char.isalpha():
```

```
        pass
```

```
    elif char.lower() in 'aeiou':
```

```
        vowels += 1
```

```
    else:
```

```
        consonants += 1
```

```
print(f"Vowels: {vowels}, Consonants: {consonants}")
```

A) Vowels: 3, Consonants: 8

B) Vowels: 4, Consonants: 7

C) Vowels: 3, Consonants: 7

D) Vowels: 4, Consonants: 8

Correct Answer: A) Vowels: 3, Consonants: 8

Explanation: "Programming" → P,r,o,g,r,a,m,m,i,n,g. Vowels: o,a,i = 3. Consonants: P,r,g,r,m,m,n,g = 8. pass is used when character is not alphabetic — no such character here so pass never executes.

24. What will be the output of the following code?

```
for num in range(1, 6):
```

```
    match num % 2:
```

```
        case 0:
```

```
print(f'{num} Even', end=' | ')
```

case 1:

```
print(f'{num} Odd', end=' | ')
```

case _:

```
print("Unknown", end=' | ')
```

A) 1 Odd | 2 Even | 3 Odd | 4 Even | 5 Odd |

B) 1 Even | 2 Odd | 3 Even | 4 Odd | 5 Even |

C) 1 Odd | 2 Even | 3 Odd | 4 Even |

D) Error — match case not supported in loops

Correct Answer: A) 1 Odd | 2 Even | 3 Odd | 4 Even | 5 Odd |

Explanation: match-case (Python 3.10+) works like switch statement. For each num, num%2 is checked. 0→Even, 1→Odd. case _ is default case (never reached here). Loop runs for 1 to 5 printing odd/even correctly.

25 Which of the following correctly describes the difference between np.arange() and np.linspace()?

A) np.arange() generates values based on step size while np.linspace() generates fixed number of evenly spaced values between start and stop

B) np.arange() generates fixed number of values while np.linspace() generates values based on step size

C) Both np.arange() and np.linspace() generate exactly same output always

D) np.arange() works only with integers while np.linspace() works only with floats

Correct Answer: A) np.arange() generates values based on step size while np.linspace() generates fixed number of evenly spaced values between start and stop

Explanation: np.arange(start, stop, step) generates values with given step size. np.linspace(start, stop, num) generates exactly num evenly spaced values. arange excludes stop, linspace includes stop by default.

26.What will be the output of the following code?

```
import numpy as np
```

```
a = np.array([10, 20, 30, 40, 50])
```

```
b = np.zeros((2, 3), dtype=int)
```

```
c = np.ones((2, 3), dtype=int)
```

```
print(a[1:4].sum(), b.shape, (b + c).sum())
```

A) 90 (2,3) 5

B) 90 (2,3) 6

C) 60 (2,3) 6

D) 90 (3,2) 6

Correct Answer: B) 90 (2,3) 6

Explanation: $a[1:4] = [20, 30, 40] \rightarrow \text{sum} = 90$. $b.\text{shape} = (2, 3)$. $b+c =$ all ones matrix of shape $(2, 3) \rightarrow \text{sum} = 2 \times 3 = 6$.

27.What will be the output of the following code?

```
import numpy as np
```

```
a = np.linspace(0, 100, 5)
```

```
print(a)
```

A) [0. 20. 40. 60. 80.]

B) [0. 25. 50. 75. 100.]

C) [0. 20. 40. 60. 100.]

D) [0. 25. 50. 75. 80.]

Correct Answer: B) [0. 25. 50. 75. 100.]

Explanation: linspace(0, 100, 5) generates exactly 5 equally spaced values between 0 and 100 inclusive. Spacing = $100/4 = 25$. So values are 0, 25, 50, 75, 100. Unlike arange, linspace always includes the stop value.

28. What will be the output of the following code?

```
import pandas as pd
```

```
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],  
        'Age': [25, 30, 35, 28],  
        'Score': [85, 90, 78, 92]}
```

```
df = pd.DataFrame(data)
```

```
print(df[df['Score'] > 85]['Name'].values)
```

- A) ['Alice' 'Bob' 'David']
- B) ['Bob' 'Charlie' 'David']
- C) ['Bob' 'David']
- D) ['Alice' 'Charlie']

Correct Answer: C) ['Bob' 'David']

Explanation: Filter rows where Score > 85 → Bob(90) and David(92) satisfy condition. Alice(85) is not strictly greater than 85. Charlie(78) < 85. .values returns numpy array of names.

29. What will be the output of the following code?

```
import pandas as pd
```

```
df = pd.DataFrame({  
    'City': ['Hyd', 'Delhi', 'Hyd', 'Mumbai', 'Delhi'],
```



```
'Sales': [100, 200, 150, 300, 250]})
```

```
result = df.groupby('City')['Sales'].sum().reset_index()
print(result.sort_values('Sales', ascending=False)
      ['City'].values[0])
```

- A) Hyd
- B) Delhi
- C) Mumbai
- D) Error

Correct Answer: B) Delhi

Explanation: groupby sums Sales → Delhi=200+250=450, Mumbai=300, Hyd=100+150=250.
sort_values descending → Delhi(450) is highest. values[0] = 'Delhi'.

30. What will be the output of the following code?

```
import pandas as pd
```

```
df = pd.DataFrame({
    'Color': ['Red', 'Blue', 'Red', 'Green'],
    'Price': [100, 200, 150, 300]})
```

```
df_encoded = pd.get_dummies(df, columns=['Color'])
print(df_encoded.shape, df_encoded.columns.tolist())
```

- A) (4, 4) ['Price', 'Color_Blue', 'Color_Green', 'Color_Red']
- B) (4, 5) ['Price', 'Color_Blue', 'Color_Green', 'Color_Red']
- C) (4, 3) ['Color_Blue', 'Color_Green', 'Color_Red']

D) (4, 4) ['Color', 'Price', 'Blue', 'Red']

Correct Answer: A) (4, 4) ['Price', 'Color_Blue', 'Color_Green', 'Color_Red']

Explanation: `pd.get_dummies()` performs one-hot encoding on 'Color' column. Color has 3 unique values (Red, Blue, Green) → creates 3 binary columns. Original Color column is dropped. Final df has Price + 3 dummy columns = 4 columns. Shape = (4,4).

31. What will be the output of the following code?

```
import pandas as pd
```

```
import numpy as np
```

```
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],  
        'Score': [85, np.nan, 78, 92]}
```

```
df = pd.DataFrame(data)
```

```
df['Score'].fillna(df['Score'].mean(), inplace=True)
```

```
print(round(df['Score'].sum(), 1))
```

A) 255.0

B) 350.0

C) 340.0

D) 256.7

Correct Answer: C) 340.0

Explanation: mean of Score = $(85+78+92)/3 = 85.0$. `fillna` replaces NaN (Bob) with 85.0. Final scores = [85, 85, 78, 92]. sum = $85+85+78+92 = 340.0$.

32. Which of the following is the CORRECT first step in Exploratory Data Analysis?

A) Build a machine learning model directly on raw data

- B) Understand the dataset — check shape, data types, missing values and basic statistics
- C) Apply one-hot encoding to all columns immediately
- D) Split data into train and test sets before any analysis

Correct Answer: B) Understand the dataset — check shape, data types, missing values and basic statistics

Explanation: The first step of EDA is always to understand the dataset using:

- `df.shape` → dimensions
- `df.dtypes` → data types
- `df.isnull().sum()` → missing values
- `df.describe()` → basic statistics

33. What is the primary purpose of a heatmap in Seaborn?

- A. To display the frequency distribution of a single variable
- B. To visualize relationships or correlations between variables using colors
- C. To create a line graph showing trends over time
- D. To detect duplicate rows in a dataset

Correct Option:

B. To visualize relationships or correlations between variables using colors

Explanation:

A heatmap uses color intensity to represent values in a matrix, commonly used to visualize correlation matrices during Exploratory Data Analysis (EDA).

34. Why are visualizations created using Matplotlib important during data analysis?

- A. They automatically clean missing values from the dataset.
- B. They help identify patterns, trends, distributions, and outliers in the data.
- C. They convert categorical variables into numerical variables.
- D. They reduce the size of the dataset.

Correct Option:

B. They help identify patterns, trends, distributions, and outliers in the data.

Explanation:

Matplotlib visualizations make it easier to understand data characteristics, detect anomalies, and communicate insights effectively before building machine learning models.

35.What is the primary purpose of a box plot in data analysis?

- A. To show the correlation between two numerical variables
- B. To visualize the distribution of data, identify the median, quartiles, and detect potential outliers
- C. To display the frequency of categorical variables
- D. To show trends in data over time

Correct Option:

B. To visualize the distribution of data, identify the median, quartiles, and detect potential outliers

Explanation:

A box plot summarizes a dataset using the **minimum value, first quartile (Q1), median (Q2), third quartile (Q3), and maximum value**. It is particularly useful for detecting **outliers** and understanding the spread and skewness of the data.